

AI PROJECT · SPRING 2026

Draw & Solve

Handwritten Math Expression Recognition

Name	Student ID	Task
Matti Hannes Vincent Lehmann	T14H06309	Presentation Slides
David Axel Spickenheuer	T14H06329	Data Preparation & Augmentation
Jonas Schenke	T14H06307	CNN Training & Evaluation
Philip Nguyen	91499153X	Presentation Slides
Staniya Thomas	T14H06310	Symbol Recognition Pipeline
Abhirup Sain	T14H06318	Streamlit UI & Integration

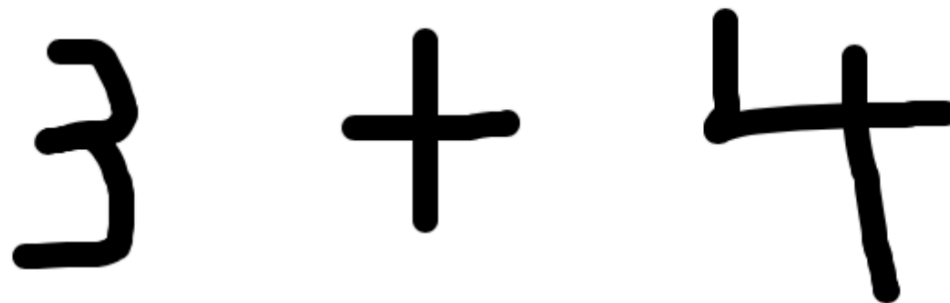
- **Handwritten maths** on tablets and touch screens is increasingly common (online exams, digital whiteboards, teaching apps)
- **Manual typing** of expressions into a calculator or CAS is slow and error-prone
- **Goal:** draw an expression once and obtain an instant, reliable numerical result

Why now? Tablet adoption in education has grown rapidly — a seamless, pen-first math solver removes a major friction point for students and teachers.



- Handwriting varies widely per writer
- Operators share features with digits (e.g. $\times$ vs x, $-$ vs $\div$)
- Multi-stroke digits confuse naive segmenters

Goal: Draw a math expression on screen $\rightarrow$ get the result instantly.



1. Draw an expression on a browser canvas
2. OpenCV segments individual symbols
3. CNN classifies each symbol (14 classes)
4. Safe AST evaluator computes the result

Digits

0 · 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9

Operators

+ · − · × · ÷

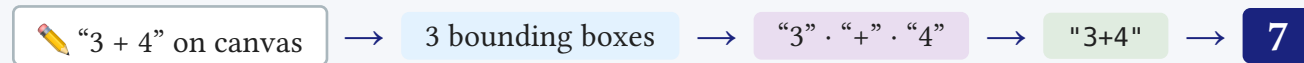
The screenshot displays the 'Draw & Solve' application interface. At the top, under 'Detected symbols', are three colored boxes: a green box with '3', a blue box with '+', and a green box with '4'. Below this, the 'Expression' field shows '3+4' and the 'Result' field shows '7'. The bottom section shows a detailed view of the detected symbols: a handwritten '3' with a green border and '3 99%' above it, a handwritten '+' with an orange border and '+ 76%' above it, and a handwritten '4' with a green border and '4 89%' above it. A legend at the bottom states: 'Detected symbols (green = digit · teal = operator · orange = uncertain)'.

Aspect	v1	v2
Classifier	MNIST CNN + geometry heuristics	Unified 14-class CNN
Dataset	MNIST only	Sagyamthapa (Kaggle)
Accuracy	99% digits · operators fragile	99.44% all 14 classes

One model for everything — no hand-crafted rules for operators, just learned features.



Example — user draws “3 + 4”:

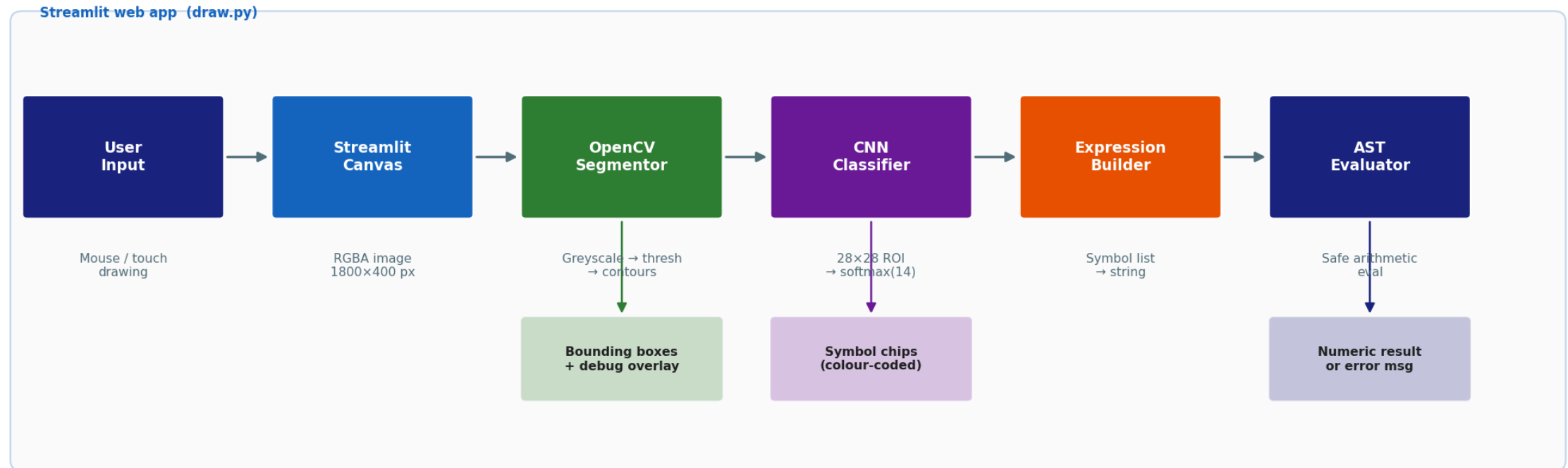


 **Frontend**
HTML5 Canvas · Streamlit

 **Segmentation**
OpenCV · proximity merge

 **Evaluation**
AST safe-eval · no exec()

Software Architecture — Draw & Solve



Model: outputs/best_model.keras · Classes: outputs/class_names.json

Frontend

Streamlit · HTML5 Canvas

Backend

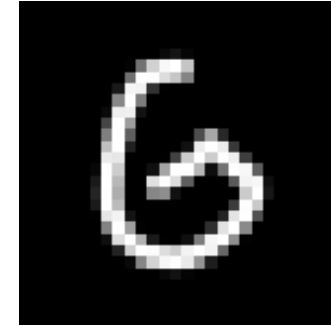
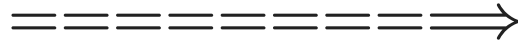
Python · OpenCV · TensorFlow

Model

Keras · 14-class CNN



How to get from the drawing to the CNN-input?



First step: Binarisation

- Canvas: white (255), ink ≈ 0
- THRESH_BINARY_INV: ink $\rightarrow 255$, bg $\rightarrow 0$
- 3×3 dilation closes gaps between strokes
- findContours detects white regions on black

6 ÷ 3

6 ÷ 3

6 ÷ 3

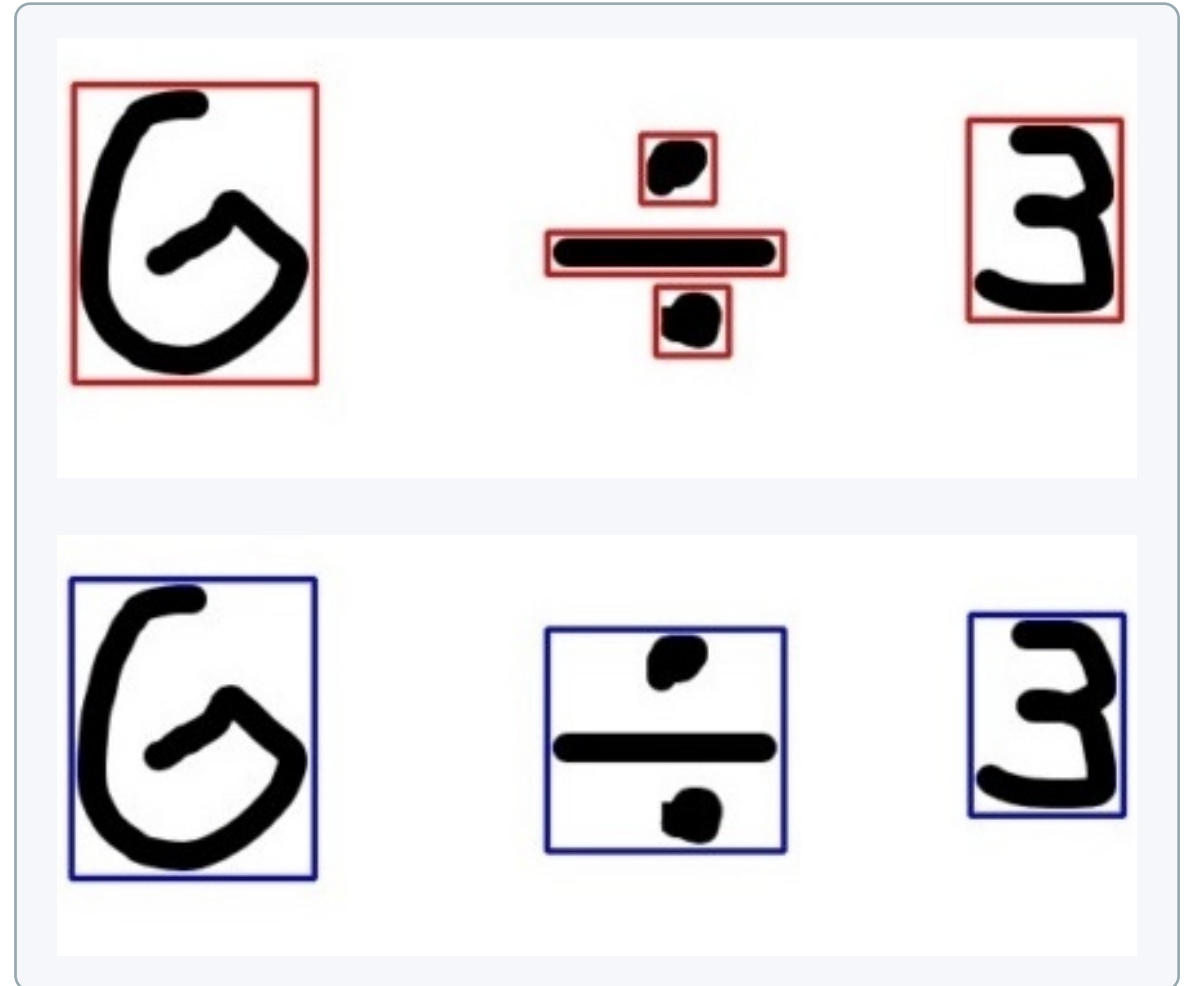
Bounding boxes

- Outermost contours only (RETR_EXTERNAL)
- Box area $< 30 \text{ px}^2$ discarded (noise filter)
- Sort all boxes left $\rightarrow$ right by x-coordinate

Problem: signs and digits can be multi-stroke ($\div$, 5, 7)
 $\rightarrow$ each stroke gets its own box

Solution: merge boxes with gap $\leq 20 \text{ px}$

- Iterate boxes left $\rightarrow$ right
- If next box starts within 20 px, expand current box



Final Step: Conversion of boxed signs to CNN input-format

1. Cropping of Box-content
2. Add padding
3. Downscaling of image to fit in 28 x 28 format
4. centering + additional padding



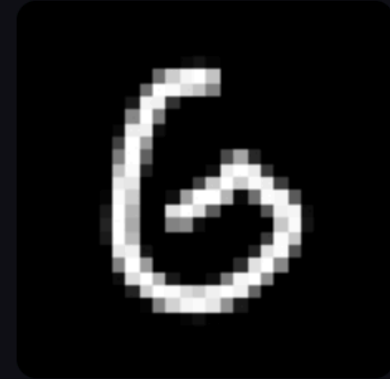
1. Cropped ROI



2. Padding Added

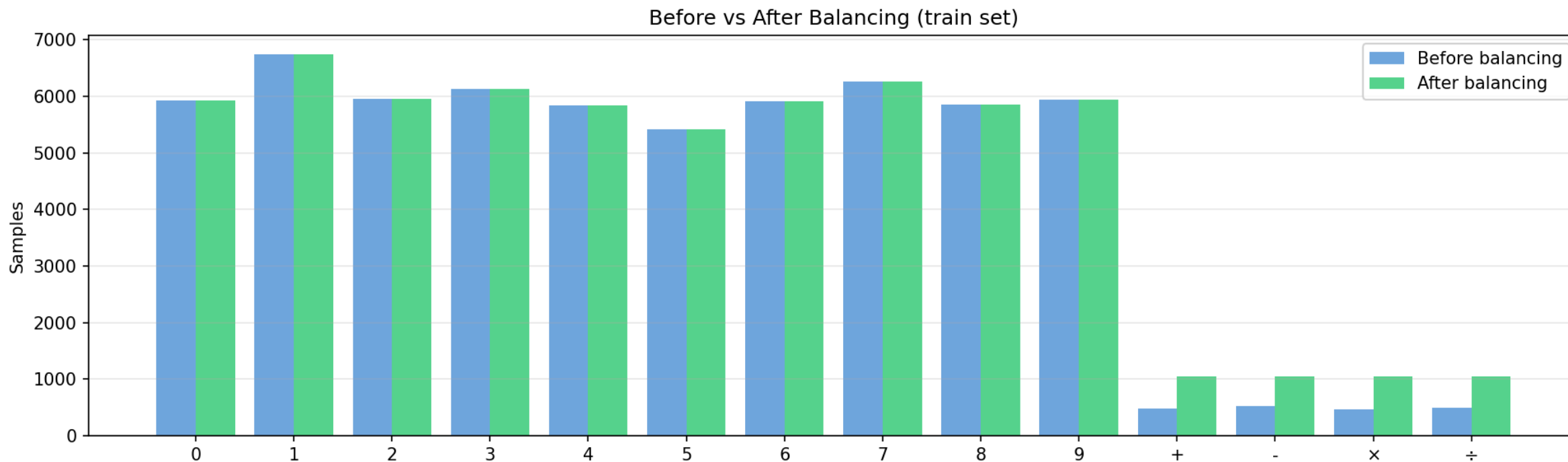


3. Scaled Down



4. Centered in 28x28

Source	Detail	Classes used	
Digits 0–9	MNIST (LeCun et al.)	Balanced per class	14 (digits + operators)
Operators $+-\times\div$	Sagyamthapa (Kaggle)	Total training pool	1 048 samples
		Train / Val	14 672 images
		Test set (held-out)	85 % / 15 % of pool
		– MNIST test	10 490 images
		– Kaggle ops (20 %)	10 000 images
			490 images (seed = 42)



Imbalance problem

- Digits: 5 000 – 6 700 samples each
- Operators: 1 048 samples each → **6:1 ratio**
- Without fix: model predicts digits almost always

Fix: downsample digits to 1 048 per class

Result: perfectly balanced — $1\,048 \times 14 = 14\,672$ images

Augmentation (training only, fill = black)

Transform	Range
Rotation	$\pm 8^\circ$
Zoom	$\pm 10\%$
Translation	$\pm 8\% \text{ x/y}$

- Simulates real writing variation
- Fill = **0 (black)** — matches MNIST convention
- Key to generalizing from only 1 048 samples